📘 SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project Title

Advanced Resume Screening System using NLP and Machine Learning

---

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to define the functional and non-functional requirements of the **Resume Screening System**, an advanced NLP-based web application that automatically evaluates, ranks, and compares resumes against a job description using **TF-IDF, Cosine Similarity, and BERT** models.

This system is designed as an **industry-grade intelligent recruitment support tool** developed using Agile methodology.

1.2 Scope

The Resume Screening System aims to:

- Automate resume screening and ranking
- Reduce manual HR effort
- Improve accuracy and consistency in resume evaluation
- Demonstrate advanced NLP, ML, and Agile practices

The system supports:

- Resume upload (PDF)
- Job description input
- NLP preprocessing
- Similarity computation using classical and transformer-based models
- Resume ranking and comparison
- Web-based interface
- Cloud deployment on Render

1.3 Definitions, Acronyms, and Abbreviations

Term	Description
NLP	Natural Language Processing

Term	Description
TF-IDF	Term Frequency–Inverse Document Frequency
BERT	Bidirectional Encoder Representations from Transformers
JD	Job Description
API	Application Programming Interface
Agile	Iterative software development methodology

1.4 Technologies Used

- Python 3.9+
- Flask (Backend Framework)
- Scikit-learn
- Sentence-Transformers (BERT)
- HTML, CSS
- Git & GitHub
- Jira (Agile Planning)
- Render (Cloud Deployment)

---

2. Overall Description

2.1 Product Perspective

The system is a **standalone web application** built with modular architecture and designed following Agile principles. It integrates NLP pipelines, ML models, and a web interface to deliver intelligent resume screening.

2.2 Product Features

- Upload and validate multiple PDF resumes
- Extract text from resumes
- NLP preprocessing (cleaning, lemmatization)
- Similarity matching using TF-IDF + Cosine Similarity
- Semantic matching using BERT
- Resume ranking and comparison
- Web-based UI

- Cloud deployment and monitoring

2.3 User Classes and Characteristics

User	Description
Recruiter / HR	Screens resumes using job description
Admin	Maintains system and deployment
Evaluator	Reviews system output and reports

2.4 Operating Environment

- Web Browser (Chrome, Edge, Firefox)
- Server: Render Cloud Platform
- Operating System: Windows / Linux
- Internet connectivity required

2.5 Design and Implementation Constraints

- Resumes must be in readable PDF format
- English language text only
- Cloud memory and compute limitations
- BERT inference time constraints

2.6 Assumptions and Dependencies

- Uploaded resumes follow standard resume formatting
- Job description is relevant and complete
- Required Python libraries are available
- Stable internet connection

---

3. System Architecture

3.1 High-Level Architecture

Client (Browser)

↓

Flask Web Server

↓

Resume Ingestion & NLP Pipeline

↓

ML & Similarity Engine

↓

Ranking & Evaluation Module

↓

Results Display

3.2 Modular Breakdown

a) Resume Ingestion Module

- Upload resumes
- Validate file type and size

b) Text Extraction Module

- Extract text using PDF parsing libraries

c) NLP Preprocessing Module

- Lowercasing
- Stopword removal
- Tokenization
- Lemmatization

d) ML & Similarity Engine

- TF-IDF vectorization
- Cosine similarity calculation
- BERT semantic similarity
- Model comparison (Spike-based)

e) Ranking & Evaluation Module

- Sort resumes by similarity score
- Generate ranked output

f) Web Application Module

- Flask APIs

- Frontend UI
- Error handling

g) Deployment & Monitoring Module

- Render deployment
- Logging
- Testing

---

4. Functional Requirements

FR-1 Resume Upload

- The system shall allow users to upload multiple PDF resumes.
- The system shall reject invalid file formats.

FR-2 Job Description Input

- The system shall accept job description text input.

FR-3 Resume Text Extraction

- The system shall extract readable text from uploaded resumes.

FR-4 NLP Preprocessing

- The system shall clean and normalize resume and JD text.

FR-5 Similarity Computation

- The system shall compute similarity using TF-IDF + Cosine Similarity.
- The system shall compute semantic similarity using BERT embeddings.

FR-6 Resume Ranking

- The system shall rank resumes in descending order of relevance.

FR-7 Results Display

- The system shall display similarity scores and rankings.

---

5. Non-Functional Requirements

5.1 Performance

- The system should process up to 10 resumes per request efficiently.
- BERT inference should complete within acceptable time limits.

5.2 Security

- Uploaded resumes should not be stored permanently.

- No user data leakage.

5.3 Usability

- Simple and intuitive UI.
- Minimal user interaction required.

5.4 Reliability

- The system should handle invalid inputs gracefully.
- Error messages should be meaningful.

5.5 Scalability

- Modular design allows future enhancements.
- Cloud-deployable architecture.

---

6. External Interface Requirements

6.1 User Interface

- Web-based interface using HTML and CSS
- Upload form and results page

6.2 Software Interfaces

- Flask APIs
- NLP and ML libraries

---

7. Agile Methodology Adoption

7.1 Agile Planning

- Project managed using **Jira**
- Work divided into **Epics, Stories, Spikes, and Tasks**
- Story points estimated using **Fibonacci series**

7.2 Version Control

- GitHub repository
- Feature-based branching strategy

---

8. Deployment Requirements

8.1 Deployment Platform

- Render Cloud Platform

8.2 Deployment Process

1. Build Flask application
2. Add requirements.txt
3. Configure Gunicorn
4. Deploy via GitHub-Render integration

---

9. Future Enhancements

- Skill extraction and highlighting
- Resume categorization
- Database integration
- Multi-language resume support
- ATS integration

---

10. Conclusion

The Advanced Resume Screening System demonstrates the application of **classical NLP techniques and modern transformer-based models** within an Agile, cloud-deployed architecture. The system is scalable, efficient, and suitable for real-world recruitment automation.